

Session 15 Slides

Quantum Graph Neural Networks

1. Why QGNNs and where they fit in hybrid workflows
2. Concrete encoding strategies (ego-graphs, QPEs, angle/amplitude)
3. Message-passing PQC blocks with shared parameters (design + cost)
4. Hybrid architectures (node/edge/graph tasks) and practical code (EstimatorQNN + TorchConnector)
5. Scalability & complexity accounting (shots, evals, params)

Motivation & recap — why QGNNs now?

QCNN → **QGNN**: QCNNs taught us *local receptive fields + parameter sharing*. QGNNs generalize this to irregular graphs by operating over *ego-graphs* (node + a fixed number of neighbors). Hybrid pipelines let us keep quantum circuits small while leveraging classical aggregation for variable graph sizes.

ACM_QGNN_Survey

When to try QGNNs: small-to-moderate graphs, problems with rich local quantum-like correlations (chemistry, physics), few-shot regimes, or when you want a compact feature extractor for a classical GNN. The recent survey shows QPEs and quantum kernels can improve downstream baselines moderately on some benchmarks (ZINC, MNIST variants) — promising but cautious.

ACM_QGNN_Survey

What is a QGNN (operational view)

A QGNN layer implements a message-passing step where each node's neighbourhood (an *ego-graph* of up to K nodes) is encoded into a small PQC, processed with a shared parameterized unitary $U(\boldsymbol{\theta})$, and measured to produce local embeddings. These embeddings are aggregated classically and optionally passed into further quantum/classical layers.

1. Gather up to $K - 1$ neighbours $\rightarrow$ form ordered vector $x_v^{\text{ego}} \in \mathbb{R}^K$ (pad if needed).
2. Encode x_v^{ego} into K qubits via angle/amplitude/basis encoding.
3. Apply the *shared* PQC $U(\boldsymbol{\theta})$ (same across nodes).
4. Read out expectations/probabilities $\rightarrow$ local feature h_v .
5. Classical aggregation $\rightarrow$ new node or graph-level features.

Key design choices: K (ego size), encoding, entangling topology, measurement mapping, optimizer, shot budget.

2) Encoding graphs into qubits — patterns & tradeoffs

A. Ego-graph construction (practical)

- Build radius-1 ego centered at node v : center + up to $K - 1$ neighbours. If degree $> K - 1$ sample deterministically (top-degree, top-centrality) or randomly with a fixed seed. Record a mask so the model can learn to ignore padded slots. (See Appendix exercise #1.)
- Deterministic ordering helps reproducibility; follow up with permutation-invariant aggregator (mean, sum, attention) to restore invariance.

B. Encoding options

1. **Angle encoding (recommended default):** one scalar $\rightarrow$ $R_Y(\alpha x)$ or R_Z . Scale features to $[-\pi, \pi]$. Simple, hardware-friendly.
2. **Amplitude encoding:** compact (log-qubit) but costly state preparation; only use when K is tiny and you have an efficient state-prep routine. Amplitude encoding is expressive but depth-expensive.
3. **Basis / binary encoding:** when features are categorical/binary.
4. **Quantum positional encodings (QPEs):** use quantum walks / Hamiltonian evolution to generate positional features per node; can capture long-range structure and improve downstream performance when concatenated with classical features (Thabet et al.). Use as *precomputed* augmentation or hybrid insertion.

C. Multi-dim features & re-uploading

- For multi-dim node features compress to 1–2 scalars per node (MLP/PCA) or use **data re-uploading**: interleave multiple encodings across layers to increase expressive power at fixed qubit count. Re-uploading costs more evaluations/gradients (pay attention to shot budget).

3) Message-passing PQC block — principled design

A. Minimal, NISQ-friendly filter (recommended)

- **Qubits:** K (ego size), where K = center + neighbors (typical 3–8 on current hardware).
- **Encoding:** angle encode K scalars via $R_Y(\phi_i)$ on qubit i .
- **Ansatz / filter $U(\theta)$:** TwoLocal-style block (RY + RZ rotations + CX entanglers), 1–2 reps. Prefer star/center-to-neighbour entanglement for message passing (control = center).
- **Readout:** expectation $\langle Z_0 \rangle$ on center qubit plus optional aggregated neighbor expectations or pairwise correlations if resources allow.
- **Parameter sharing:** same θ across all ego-graphs (like GNN layer weights).

Tradeoffs: more entanglers $\uparrow$ expressivity but $\uparrow$ two-qubit error & SWAPs (if hardware topology mismatches).

B. QAOA as message operator (when appropriate)

- QAOA layers can encode combinatorial interactions (cost Hamiltonian from local edges). They are natural for graph-structured combinatorial tasks (community detection, partitioning) but require careful parameter handling and shallow depths on NISQ devices. Use pretraining or fixed schedules to reduce tuning overhead.

C. Quantum kernels and swap-test style embeddings

- If gradients collapse or you prefer non-parametric similarity, compute quantum kernel overlaps between ego-embeddings (swap test or shallow overlap estimation) and feed into SVM / classical kernel method. Quantum kernel methods can be powerful but scale as $O(N^2)$ kernel entries and need more measurements; survey lists QBMK, QKSAN and covariant kernels as representative techniques.

4) Hybrid architectures — patterns & templates

A. Node-level QGNN (hybrid)

- Per node: PQC filter $\rightarrow$ local embedding h_v (scalar or vector).
- Classical aggregator: mean / attention over neighbor embeddings.
- Classical MLP head: per-node prediction (classification/regression).

B. Graph-level (molecule/property)

- Apply PQC per node (or per functional block), aggregate node embeddings (sum/attention), pass through MLP.

C. Edge scoring / link prediction

- Build joint ego-graph for $(\text{center} = (u,v) \text{ combined})$ and run PQC to produce an edge score.

D. Pretraining & warm starts

- Pretrain classical GNN, use its weights or embeddings to initialize PQC parameters (warm start) — often hybrid pretraining reduces quantum depth and improves convergence.

Pitfalls/tips and tricks

It's the same as the last two weeks. It's a quantum circuit in an NN., Increase shot count as you go, consider starting gradient free then switching. Keep your circuits small...